Constant-Shape LLM Workflows: Verified Elimination of Usage-Metadata Side Channels

NAMBI RAJAN M, Vellore Institute of Technology, India
RANJITHKUMAR S, Vellore Institute of Technology, India

Observability platforms, gateways and billing systems record the usage of every call a large language model (LLM) workflow makes—which model, how many input and output tokens, how long it took—even when prompt and response content is hidden, as their privacy modes recommend. We show that this metadata is a side channel whenever a workflow's control flow depends on private content, and that such dependence is the norm: 37 of 54 (69%) application workflows in the official LangGraph, LangChain and Prompt Flow examples route, loop or fan out on content. On faithful ports of nine of these workflows running real models, a classifier given only content-free usage records recovers the workflow's secret routing decision with 25% median balanced accuracy (chance 25%).

We present OrchLang, a workflow language and compiler that closes this channel by construction. Information-flow labels identify calls whose usage could depend on secrets; tokenizer-exact static bounds size them; and a shaping transformation aligns the call sequences of secret branches into a minimum-cost common supersequence, inserts dummy calls, pads every secret-dependent request to an exact token count and forces outputs to a fixed length. An executable validator checks every shaped program, and a Lean 4 proof establishes that validated programs emit identical usage traces for all secrets—pointwise for every randomness tape and hence in distribution—and compute the original workflow's results when padding is transparent. On the nine workflows, shaped programs produce one constant usage trace per workflow (attack accuracy equals the majority rate), cost 1.6× the undefended tokens at the median, and preserve task accuracy within 2.9 points. Alignment is 0% cheaper than executing all branches, and the validator rejected every one of 89 injected compiler faults that an independent oracle showed to leak.

Additional Key Words and Phrases: side channels, information flow, noninterference, LLM agents, workflow compilers, translation validation, Lean

1 INTRODUCTION

Applications built on large language models are increasingly *workflows*: graphs of model calls, tools and glue code in which one call's output decides what happens next. A router classifies a support ticket and dispatches it to a billing or a technical chain; a grader decides whether retrieved documents are relevant and, if not, triggers a web search; an evaluator rejects a draft and sends it back for another round. The LangGraph documentation teaches these patterns as its core workflow idioms, and Prompt Flow encodes them as conditionally activated nodes.

Every call such a workflow makes leaves a usage record. Providers bill for input and output tokens per call; gateways log the served model and token counts; observability platforms export per-call spans. Operators who handle sensitive content are told to redact it: LangSmith can hide inputs and outputs while "the trace produced will have its metadata present" [30]; Helicone's omit-logs mode drops request bodies but "you can still see the key metrics for each request"—cost, latency and token usage [24]; the OpenTelemetry GenAI conventions make message content opt-in but recommend recording the model and the token counts of every call [42]. The redacted trace is then shared with an observability vendor, a gateway operator, a finance team or a security information system.

This paper starts from a simple observation: *when control flow depends on content, the content-free usage trace is a function of the content.* A support ticket routed to human review makes one call, a

Authors' addresses: Nambi Rajan M, Department of Computational Intelligence, SCOPE, Vellore Institute of Technology, Vellore, India, nambirajan.m2024@vitstudent.ac.in; Ranjithkumar S, Department of Analytics, SCOPE, Vellore Institute of Technology, Vellore, India, ranjithkumar.s@vit.ac.in.

ticket routed to drafting makes two; a question the private corpus cannot answer triggers a rewrite and another grading round; a "medical" request is answered by a longer chain than a "billing" one. None of this is hidden by encryption or redaction, because the leak is in the *shape* of the computation, not in its bytes. Recent measurements confirm the danger in adjacent settings: logged router decisions reveal sensitive request topics on real traffic [12], encrypted traffic fingerprints agent workflows [44, 57], and an LLM classifier's output length reveals its class [60]. Agent security frameworks based on information-flow control explicitly leave this channel open: FIDES notes that "an adversary that is able to see the sequence of tool calls may still be able to infer" secrets [15], and CaMeL lists side channels as an open problem [18].

How common is the problem? We coded every distinct application workflow in the official LangGraph, LangChain documentation and Prompt Flow examples at pinned revisions (section 2.3). Of 54 application workflows, 37 (69%) branch, loop or fan out on content, and in 35 of those the decision depends on what the user wrote. In 22 of 25 looping workflows the loop has no iteration bound at all. The channel is not an exotic corner case; it is the default behavior of the idioms these frameworks teach.

Why existing mitigations do not suffice. Deployed mitigations pad streamed packets with random bytes [33, 41]; they do not touch usage fields, which are logged unpadded, and random padding leaves measurable residual leakage even at the network level [32, 34, 58]. Padding a single call's output to its maximum length [60] hides that call's length but not *which* calls a workflow made, how many, or with which model. Constant-time programming has long solved the analogous problem for CPU instructions by transforming secret branches so that both execute the same operations [1, 10, 38, 40], but LLM calls are not instructions: their cost is set by a tokenizer and by stochastic generation, the observer sees per-call token counts, and a "dummy" operation is a billed model call.

This paper. We present OrchLang, a language and compiler that makes the usage trace of an LLM workflow independent of its secrets, together with machine-checked guarantees and an evaluation on real models.

- **Measurement.** A census of 72 example workflows and faithful ports of nine representative ones—routing, prompt chaining, evaluator–optimizer loops, orchestrator–worker fan-out, a Prompt Flow switch, an email-support agent, corrective and adaptive retrieval-augmented generation, and a calculator agent—run with Llama 3.2 models over public datasets. From content-free usage records alone, a classifier recovers the workflow's internal secret decision in all nine workflows (section 7.2).
- **Detection.** OrchLang labels data with a two-point confidentiality lattice, infers which calls run under secret control or send secret-derived requests, and reports each construct through which a secret can change the usage trace (section 3).
- **Repair.** A shaping transformation (section 4) pads secret-dependent requests to a tokenizer-exact static bound, forces their outputs to a fixed length, aligns the call sequences of secret branches into a minimum-cost common supersequence filled with dummy calls of the matching model, and pads secret-guarded loops to their bound. Only calls whose usage can depend on a secret are padded.
- **Verification.** The compiler is untrusted. A small validator, proved sound in Lean 4 without additional axioms, accepts a shaped program only if it satisfies a padding discipline and only adds padding to the original program (section 5). We prove that accepted programs emit the same usage trace for all secret inputs, for every randomness tape and therefore in

distribution, and compute the original program's results whenever padding is transparent. Both assumptions are stated as explicit hypotheses and tested empirically.

- **Evaluation.** Shaped programs collapse each workflow to a single observable trace, so attacks fall to the majority rate, at a median token overhead of 1.6× and with task accuracy within 2.9 points of the undefended workflow (section 7). Ablations corresponding to deployed or proposed defenses—random padding at equal budget, per-call padding without alignment [60], and input-only padding on APIs that cannot force output length—all leak. Fault injection shows that the validator catches every leaking compiler bug we injected.

What is and is not claimed. The guarantee concerns an observer of per-call usage records: model identity, input and output token counts, and tool names. It is exact for that observer under the stated conformance assumptions, which self-hosted serving systems satisfy through exact tokenizers and forced-length decoding (`ignore_eos`, `min_tokens`) [22, 50]. Wall-clock latency is not covered by the theorem; we measure it separately (section 7.6). Hosted APIs that cannot force output length support only a weaker input-and-shape padding whose residual leakage we quantify. Padding costs tokens, and we report that cost honestly; the transformation pays it only where secrets can flow.

All code, proofs, workflow ports, datasets, raw run records and analysis scripts are released with the paper (section 10).

2 USAGE METADATA AS A SIDE CHANNEL

2.1 Setting and observer

A *workflow* receives inputs, some of which are confidential (a user's message, a private document, a ticket), and issues a sequence of model calls and tool invocations whose requests, order and number may depend on earlier results. We consider an observer who sees, for each execution, the *usage trace*

$$O \;=\; e_1\, e_2 \cdots e_n, \qquad e_i \in \{\mathsf{llm}(m, t_{\mathrm{in}}, t_{\mathrm{out}}),\ \mathsf{tool}(f)\},$$

the ordered list of model calls with their model identifier m and their billed input and output token counts, together with the names f of invoked tools. The observer does not see prompts, responses or tool arguments. This is precisely the information retained by content-redacting telemetry: the OpenTelemetry GenAI span conventions mark message content as opt-in but recommend the request and response model and the input and output token counts on every span [42]; LangSmith's hide-inputs/outputs mode keeps trace metadata [30]; Helicone's omit-logs mode keeps cost, latency and token usage [24]; Langfuse masking rewrites strings but leaves counts and the observation tree [31]; and gateways commonly log the served model and token counts with content logging disabled [12]. Parties who see such traces include observability vendors and their sub-processors, gateway and platform operators, finance teams reading per-request bills, and anyone with read access to a shared tracing project. A network observer sees a coarser projection through request and response sizes [54, 57].

The goal is *usage noninterference*: two executions whose public inputs agree must produce traces with the same distribution, whatever their secret inputs. Because generation is stochastic we state the property per randomness tape, which implies the distributional version (section 5).

Out of scope. Wall-clock latency is correlated with token counts and is measured but not covered by the theorem (section 7.6). Streaming-granularity channels (per-token packet sizes and timing) [34, 53, 54], prompt-cache timing and cached-token counts [23, 55], provider-internal state and hidden reasoning tokens are orthogonal; section 8 discusses how they compose with our defense. We assume the provider bills honestly, tokenizes deterministically and honors length controls; billing-integrity

audits address the converse problem. The observer is passive: we do not consider adversaries who choose the secret inputs adaptively, although the guarantee holds for every input.

2.2 How content reaches the trace

Content influences the usage trace in four ways, all of which occur in the workflows below.

Request size. A request containing secret text has a secret-dependent input token count, and a response to it has a secret-dependent output token count. This is the single-call channel studied by Zhang et al. [60].

Branching. A guard on secret-derived data selects between arms that make different calls, possibly to different models. The number of calls, their order and their models reveal the branch; Chen's router logs are the one-decision special case [12].

Loops and fan-out. Iteration counts that depend on a grader's verdict, on whether a tool call was emitted, or on the length of an LLM-produced list reveal that verdict or length.

Data flow. A response produced under secret control, or computed from secret data, becomes part of later requests and so propagates the dependence to calls that are not themselves inside a secret branch.

2.3 A census of example workflows

To estimate how often these patterns occur we coded every distinct example workflow in three widely used sources at pinned revisions: the LangGraph repository [29], the LangChain/LangGraph documentation [28] and the Prompt Flow repository [37]. Candidates were found by searching for graph constructs (`StateGraph`, conditional edges, `Command(goto=...)`, `Send`, Prompt Flow `activate` conditions) and model-call indicators, then reading every candidate by hand and tracing each conditional edge, loop and early return to the inputs of its predicate. Python/JavaScript, Graph/Functional API and provider variants of the same topology were merged. The unit of analysis is a distinct application workflow; evaluation flows and API demonstrations are reported separately. The full coding scheme, the per-workflow evidence with file and line numbers, and the scripts are part of the artifact.

Table 1 summarizes the result. Among 54 application workflows, 37 (69%) have content-dependent control flow: 34 branch on content, 25 loop on it and 6 fan out over content-derived lists. In 35 of the 37 the decision depends on the semantics of user-provided input. The LangGraph corpus is almost entirely content-dependent (97%), as expected from material that teaches routers, graders and agents; Prompt Flow's directed-acyclic examples are mostly straight-line. Two further observations matter for defenses. First, 22 of 25 looping workflows have no iteration bound in code and stop only on a content condition or the framework's recursion limit, so the iteration count is an unbounded-capacity channel. Second, in 5 workflows the number of calls is constant and content leaks only through token counts, so hiding the call count alone is insufficient; and at least 7 workflows assign different models to different roles, so the model field alone reveals which role ran.

3 THE ORCHLANG LANGUAGE

OrchLang is a small language for bounded LLM workflows. It is deliberately close to the idioms of the frameworks it ports from—prompts with named placeholders, model calls with a token cap, deterministic glue functions, routing on model outputs, loops until a condition—while making every quantity that affects usage explicit and bounded.

Table 1. Content-dependent control flow in example workflows (pinned revisions; manual coding with line-level evidence). A workflow may fall into several categories.

Subset	Workflows	Content-dependent	Branching	Loop	Fan-out
Application workflows (all sources)	54	37 (69%)	34	25	6
LangGraph and LangChain docs	31	30 (97%)	28	24	5
Prompt Flow	23	7 (30%)	6	1	1
Evaluation flows (Prompt Flow)	10	2	0	0	2
API-demonstration classes	8	3	2	2	1

```
workflow EmailSupport {
  input email_content: text secret max_tokens 300;
  model llm = "llama1b";
  tool email_route(x) max_tokens 2;
  tool field(x, key, fallback) max_tokens 3;
  prompt classify(email, sender) = user "Analyze␣this␣customer␣email␣and␣classify␣it:␣...";
  prompt draft(email, intent, urgency, context) = user "Draft␣a␣response␣to␣this␣customer␣email:␣
       ...";
  let cls = call classify(email_content, "customer@example.com") using llm max_tokens 80;
  let route = tool email_route(cls);
  let intent = tool field(cls, "intent", "unknown");
  let urgency = tool field(cls, "urgency", "medium");
  if route == "review" {
    let reply = "Escalated␣to␣human␣review.";
  } else {
    if route == "docs" { let context = "Relevant␣documentation:␣..."; }
    else if route == "bug" { let context = "Relevant␣documentation:\n-␣Bug␣ticket␣BUG-12345␣
         created"; }
    else { let context = ""; }
    let reply = call draft(email_content, intent, urgency, context) using llm max_tokens 320;
  }
  output reply;
}
```

Fig. 1. Port of the LangGraph email-support agent (prompts abbreviated). Billing or critical emails are escalated without a drafting call; the number of calls and the size of the draft request reveal the route.

3.1 Syntax by example

Figure 1 shows the port of the LangGraph "Thinking in LangGraph" customer-support agent [28]. Inputs carry a confidentiality label (secret or public) and a token bound. A model declaration names a model contract; a prompt is a list of chat messages whose {x} placeholders are its parameters. A call sends a prompt to a model with a visible output cap (max_tokens); a tool applies a pure, deterministic function from a library (here, the tutorial's routing logic and a field parser), with a declared output bound. Conditionals test an output for equality with or a prefix of a string, and repeat n until c runs its body while c is false, at most n times. Variables are global to the workflow; unassigned variables hold the empty string.

Lowering. The compiler lowers a workflow to a core language (fig. 2) with integer variable slots. A call's request is an expression of literals, variables and concatenation obtained by rendering the

$$
\begin{array}{rcl}
e & ::= & \texttt{lit}\ s \mid \texttt{var}\ x \mid e +\!\!+ e \\
g & ::= & x = s \mid s \sqsubseteq x \qquad \text{(equality, prefix)} \\
c & ::= & \texttt{skip} \mid c; c \mid x \leftarrow \texttt{call}_i\ m\ k\ e\ [\texttt{pad}\ T\ N] \mid x \leftarrow \texttt{tool}\ f\ e \\
 & \mid & \texttt{dummy}\ m\ T\ N \mid \texttt{dummyTool}\ f \mid \texttt{if}\ g\ \texttt{then}\ c\ \texttt{else}\ c \mid \texttt{loop}\ n\ \texttt{until}\ g\ c\ [\text{padded}]
\end{array}
$$

Fig. 2. Core language. A call has a static site i, model m, visible cap k and request e; the shaping compiler may add a padding annotation, dummy statements and the padded flag on loops. Source programs contain none of these.

prompt with the model's own chat template; we reproduce the Jinja templates of the evaluated models byte for byte, which the test suite checks against the templates embedded in the model files. The request expression is therefore exactly the text the model receives, and its literal parts can be counted with the model's tokenizer. Tool arguments are joined with a separator character; constants are a built-in tool.

3.2 Labels and detection

Labels form the two-point lattice $\mathsf{L} \sqsubseteq \mathsf{H}$. A labelling Γ assigns a label to every variable. Secret inputs are H; the compiler infers the least flow-insensitive labelling satisfying the standard rules [51]: a variable assigned by a call or tool whose argument has label ℓ, under program-counter label pc, has label at least $pc \sqcup \ell$, and the arms of a conditional or the body of a loop whose guard variable has label ℓ_g run under $pc \sqcup \ell_g$. Model responses are labelled like their requests: the provider sees content, but the observer does not, so responses are as secret as what produced them.

Unlike conventional information-flow control, OrchLang never *rejects* a flow of secrets into a prompt: the model is trusted with content. What it tracks is whether the *usage* of a call can depend on a secret. A call can leak if its request has label H (its input and output counts may vary) or if it runs under $pc = \mathsf{H}$ (whether it happens may vary); a conditional or loop can leak if its guard is H. The detector reports each such construct with its source line (table 2 lists the findings for the nine ports). A workflow with no findings already satisfies the padding discipline of section 5.2 and is accepted unchanged.

3.3 Static token bounds

Shaping needs, for each secret-dependent call, an input size T that every possible request fits into and an output size N. The output size is the call's declared cap or larger. For inputs, OrchLang computes T from the request expression: each literal piece is counted with the model's tokenizer, each variable contributes its bound—the declared bound of an input, the cap of the call that produced it, or the declared output bound of the tool—and one token of slack is added at each piece boundary because subword tokenizers are not additive over concatenation; the fixed wrapper of the padding message (section 4.1) is added on top. Bounds over loops take the maximum over assignments. The runtime enforces the bounds rather than trusting them: a tool output longer than its bound is truncated to it, and a request that would exceed T is shortened before padding. Truncation therefore affects function, never security, and our runs record how often it happens.

4 SHAPING WORKFLOWS

The shaping compiler rewrites a lowered program so that every statement that can run under secret control, or whose request is secret, has a usage footprint fixed at compile time. It has four mechanisms: exact input padding, forced output length, alignment of secret branches with dummy calls, and padding of secret loops.

4.1 Padding a call

A call $x \leftarrow \texttt{call}_i\ m\ k\ e$ is annotated with pad $T\ N$ when it runs under $pc = \mathsf{H}$ or its request label is H. At run time the request $r = [\![e]\!]$ is transformed by a function $padReq_m(T, r)$ that returns a request of exactly T tokens under model m's tokenizer, and the model is asked to generate exactly N tokens; the response stored in x is the visible part: everything before the first stop token, cut at the call's own cap $k \le N$.

Exact input padding. Our padder places a separate system message of filler units immediately before the assistant header, so the chat template's special tokens separate it from the content and token counts are additive across the boundary. It counts the full request with the model's tokenizer, chooses the number of filler units that reaches T, re-counts, and adjusts until the count is exactly T; if the unpadded request already exceeds T, it removes content tokens from the end of the last turn first. The filler unit is chosen per tokenizer so that k repetitions tokenize to k tokens; we use " .", which a pilot found least disruptive among three candidates (section 7.5). The test suite checks exactness with both tokenizers on multi-byte text.

Forced output length. Self-hosted serving systems can continue generation past the end-of-sequence token: vLLM exposes `ignore_eos` and `min_tokens` [50] and llama.cpp exposes `ignore_eos` with `n_predict` [22]. Under a fixed decoding procedure and randomness, the tokens before the first stop token are exactly those natural decoding would produce, so cutting the visible response at the first stop token and at the cap k recovers the unpadded output. We test this property directly (section 7.5). Hosted APIs that cannot force length support only input padding and alignment; we measure the leakage that remains in that mode.

4.2 Aligning secret branches

Padding makes each call's counts constant but not *which* calls happen. For a conditional whose arms run under $pc = \mathsf{H}$, the compiler first shapes each arm recursively, so that every statement in an arm has a static *shape*: a fixed sequence of events $\mathsf{llm}(m, T, N)$ or $\mathsf{tool}(f)$. It then needs a common shape for all arms. Chains $\texttt{if}\ g_1\ A\ \texttt{else if}\ g_2\ B\ \texttt{else}\ C$ are flattened so that all k arms are aligned jointly.

Two arms' event sequences are aligned by a dynamic program in the style of weighted shortest common supersequence. Events are *compatible* if both are model calls to the same model or both are the same tool; aligning two compatible calls yields a call padded to the componentwise maximum $(\max T, \max N)$; an event present in only one arm becomes a dummy in the other. The cost of a profile is the price of executing it, $\sum(w_{\text{in}}T + w_{\text{out}}N)$ over its calls with $w_{\text{out}}/w_{\text{in}} = 4$ (a typical API price ratio), and the dynamic program finds the minimum-cost supersequence of the two arms in $O(|A| \cdot |B|)$. For $k > 2$ arms the compiler aligns progressively in decreasing order of cost, remapping earlier arms' positions as the profile grows; this is a heuristic, since optimal multiple supersequence is NP-hard. Finally each arm is rebuilt against the common profile: its own calls are re-padded to the profile's maxima, and every profile position it lacks becomes a `dummy` $m\ T\ N$ (a real, billed call with a fixed padded request whose response is discarded) or a `dummyTool` f. Nested constructs that already have a static shape are aligned as atomic units.

Alignment is the LLM-call analogue of cross-copying [1] and of padding to supersequences over anonymity sets in website-fingerprinting defenses [52]. Its baselines are executing every arm's calls (secure multi-execution [19]; our *naive* strategy) and padding every call of a model to one global cell size (as in BuFLO [21]; our *uniform* strategy).

4.3 Padding loops

A loop `loop` n `until` g c whose guard is H, or that runs under $pc = \mathsf{H}$, is shaped by shaping its body under $pc = \mathsf{H}$—so the body has a static shape s—and setting its *padded* flag: when the guard becomes true after $j < n$ iterations, the loop executes $n - j$ ghost iterations that emit the dummy calls of s without touching the store. The loop therefore always emits s^n. Unbounded loops, which the census shows are common, must be given a bound when ported; the bound is part of the workflow's specification and also caps its worst-case cost.

4.4 Taint guidance

Statements whose usage cannot depend on secrets—calls with a public request outside secret control, public loops and branches—are left untouched. This matters for workflows that mix public and secret processing; in the nine ported workflows every call ultimately depends on the user's secret message, so the taint-guided strategy coincides with padding everything (section 7.4). On randomly generated programs that mix public and secret calls it lowers cost by up to 91% (section 7.4).

5 GUARANTEES

The shaping compiler is several hundred lines of untrusted Python. Rather than verify it, we follow the translation-validation approach [39, 43]: a small checker decides, for each original program and each candidate shaped program, whether the candidate is acceptable, and we prove in Lean 4 [17] that every accepted candidate has the desired properties. The development (`proof/ShapeCore.lean`, 998 lines, 28 theorems) uses only Lean's core library; `#print axioms` reports only `propext` and `Quot.sound` for the main results. The checker is the same Lean code, compiled and run on the JSON form of programs, so no separate implementation of the checker needs to be trusted.

5.1 Semantics

The world. Everything outside the program—tokenizers, models, the padder, tools—is a parameter w of the semantics (`World` in Lean), consisting of functions $tok_m(r)$ (token count of request r under model m), $gen_m(r, k, \omega)$ (natural response with cap k on randomness cell ω), $forced_m(r, \omega, N)$ (forced-length response), $view_m(k, \rho)$ (visible part of a raw response), $padReq_m(T, r)$, $outTok_m(\rho)$ and pure tool functions. No property of these functions is assumed except through the two explicit hypotheses below.

Randomness. Model randomness is a tape $\omega : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$: the j-th execution of call site i reads cell $\omega(i, j)$. For any tape distribution with independent, identically distributed cells this realizes fresh randomness per call, which is how stochastic decoding behaves; the indexing by site is a proof device that lets two executions taking different secret paths share the randomness of the calls they have in common. Dummy calls and ghost loop iterations read no cell.

Execution. The big-step function $exec_{w,\omega}(c, \sigma)$ maps a statement and a state—store, per-site counters and the usage trace so far—to a final state. An unpadded call appends $\mathsf{llm}(m, tok_m(r), outTok_m(\rho))$ for request r and response $\rho = gen_m(r, k, \omega(i, j))$ and stores ρ; a padded call sends $r' = padReq_m(T, r)$, receives $\rho = forced_m(r', \omega(i, j), N)$, appends $\mathsf{llm}(m, tok_m(r'), outTok_m(\rho))$ and stores $view_m(k, \rho)$. A dummy sends a fixed padded request and appends its event. A loop runs its body while its guard is false, at most n times; if it is padded and stops after $j < n$ iterations, it appends $n - j$ copies of its body's static shape as dummy events. The usage trace of a run is the trace component of the final state; it is exactly the observation of section 2.1.

5.2 The padding discipline

Static shape. The partial function $shape(c)$ returns the trace c must emit, if fixed: a padded call has shape $[\mathsf{llm}(m, T, N)]$, an unpadded call has none, a conditional has a shape only if both arms have the same one, and a padded loop with body shape s has shape s^n.

Discipline. The judgement $\Gamma \vdash_{pc} c$ (`wt`) is a syntax-directed check:

$$\frac{pc \sqcup \Gamma(e) \sqsubseteq \Gamma(x) \qquad pc \sqcup \Gamma(e) = \mathsf{H} \Rightarrow pad \text{ present}}{\Gamma \vdash_{pc} x \leftarrow \mathtt{call}_i\ m\ k\ e\ pad}$$

$$\frac{pc' = pc \sqcup \Gamma(g) \qquad \Gamma \vdash_{pc'} c_1 \qquad \Gamma \vdash_{pc'} c_2 \qquad pc' = \mathsf{H} \Rightarrow shape(\mathtt{if}\ g\ c_1\ c_2) \text{ defined}}{\Gamma \vdash_{pc} \mathtt{if}\ g\ \mathtt{then}\ c_1\ \mathtt{else}\ c_2}$$

$$\frac{pc' = pc \sqcup \Gamma(g) \qquad \Gamma \vdash_{pc'} c \qquad pc' = \mathsf{H} \Rightarrow padded \wedge shape(c) \text{ defined}}{\Gamma \vdash_{pc} \mathtt{loop}\ n\ \mathtt{until}\ g\ c\ padded}$$

with the evident rules for tools, sequencing and dummies. A second check (`sitesOK`) requires that the call sites reached under $pc = \mathsf{H}$ are disjoint from those reached under $pc = \mathsf{L}$, so that the randomness cells read on secret paths never coincide with cells read on public paths. A third (`capsOK`) requires $k \le N$ for every padded call.

Validation. The validator (`validateProgram`) accepts a candidate c' for the original c if (i) $\Gamma \vdash_{\mathsf{L}} c'$, (ii) the randomness-site check holds for the set of high sites computed from c', (iii) $k \le N$ throughout, and (iv) erasing all padding annotations, dummies and padded flags from both programs and normalizing sequencing and `skip` yields syntactically identical programs (`clean (erase c') == clean (erase c)`). Condition (iv) ensures the compiler only added padding; it did not reorder, drop or modify any call or tool.

5.3 Theorems

Two hypotheses on the world are stated as Lean propositions and never assumed globally.

Conformance (`World.Conforms`): $tok_m(padReq_m(T, r)) = T$ for all m, T, r, and $outTok_m(forced_m(r, \omega, N)) = N$ for all m, r, ω, N. Padding is exact and forced decoding emits exactly N tokens.

Transparency (`World.Transparent`): $view_m(k, forced_m(padReq_m(T, r), \omega, N)) = gen_m(r, k, \omega)$ whenever $k \le N$. The visible part of a padded, forced response equals the natural response to the unpadded request on the same randomness.

Theorem 5.1 (Usage noninterference; `noninterference`, `trace_noninterference`). *Assume Conformance. If $\Gamma \vdash_{\mathsf{L}} c$ and the randomness-site check holds, then for all stores σ_1, σ_2 that agree on L-labelled variables and every tape ω, the executions of c from σ_1 and from σ_2 emit identical usage traces (and end in stores that agree on L variables, with counters that agree on all public sites).*

Corollary 5.2 (Distributional form; `distribution_noninterference`). *Under the same hypotheses, for every weighted family of tapes—in particular every distribution of i.i.d. randomness cells—the induced distributions of usage traces from σ_1 and σ_2 are identical. Consequently every observer, however computationally powerful, guesses any function of the secrets with exactly its prior success probability.*

Theorem 5.3 (Functional preservation; `erase_correct`, `clean_data`). *Assume Transparency. If the caps check holds for c', then c' and $clean(erase(c'))$ end in the same store (and counters) from every initial store, for every tape.*

Theorem 5.4 (Validator soundness; `validateProgram_sound`). *If validateProgram*(Γ, c, c') *returns true, then (a) under Conformance, c' satisfies the conclusion of theorem 5.1; and (b) under Transparency, c' ends in the same store as the original program c from every initial store and tape.*

Proof structure. The central lemma (`shape_exec`) states that under Conformance a statement with a static shape s appends exactly s to the trace from every state. Two confinement lemmas show that statements accepted under $pc = \mathsf{H}$ write no L variable (`high_store`) and advance no public site's counter (`high_count`). Noninterference then follows by induction over the program for $pc = \mathsf{L}$: a call with a public request reads equal requests and equal randomness cells in both runs; a call with a secret request is padded and emits the same event; a secret conditional emits its shape in both runs whatever arm it takes; a public conditional takes the same arm; loops are handled by induction on the iteration bound. Functional preservation is a simulation between a program and its erasure that ignores the trace. The development is 998 lines including the executable JSON decoder and `#guard` regression tests that run at every check.

What the theorems do not say. They say nothing about latency, which is not part of the trace (section 7.6); nothing about providers that violate Conformance, which our runtime checks on every call; and functional preservation holds only to the extent Transparency does. Transparency is exact for the forced-output half by construction of decoding, and we test it (section 7.5); it is *false* in general for the input half, because adding a padding message changes the prompt. We therefore treat input-padding transparency as an empirical question and report task accuracy with and without shaping. The Python compiler, lowering and runtime are not verified; the validator makes the compiler irrelevant to security, and the runtime's conformance is checked on every call of every evaluation run.

6 IMPLEMENTATION

Compiler. The OrchLang front end, label inference, bound computation, detector and the four shaping strategies are implemented in Python (`orchlang/`, about 1515 lines without tests). The compiler emits both the original and the shaped core program as JSON, which the Lean validator decodes and checks; the compiler refuses to run a program the validator rejects.

Validator. The executable validator is the `checkJson` entry point of `proof/ShapeCore.lean`, run with `lean -run`. It decodes both programs, evaluates `validateProgram`, and also reports each component check and the static shape of the accepted program, which the runtime compares against every observed trace. Validating all four strategies of one workflow takes about 85.8 seconds, dominated by Lean's start-up and elaboration of the proof file; the check itself is linear in program size.

Runtime. The reference runtime interprets the core language over llama.cpp (via `llama-cpp-python` 0.3.36) on CPU. It implements the semantics of section 5.1 directly: each call site keeps an execution counter and derives its sampling seed from the run seed, the site and the counter; padded calls use the padder of section 4.1 and decode with end-of-sequence suppressed for exactly N tokens; the key–value cache is reset before every call, so there is no cross-call prompt caching. Every call produces a usage record (model, prompt tokens as counted by the serving library, generated tokens, wall-clock latency) together with analysis-only fields that the observer never sees: whether the call was a dummy, its site, and its natural length. A deterministic mock backend with the same interface is used for differential testing.

Ports. The nine evaluated workflows (table 2) were ported from the census's representative candidates. Each port copies prompts verbatim from the pinned source and records every deviation

in a header comment and a per-workflow specification: structured outputs (Pydantic schemas, tool calling) become constrained text answers parsed by deterministic tools, live retrieval and web search become BM25 retrieval over indexed and held-out SQuAD paragraphs, unbounded loops receive explicit bounds, human-in-the-loop interrupts become a fixed escalation reply, and parallel fan-out runs sequentially. These deviations change absolute token counts relative to the originals but preserve their control structure, which is what determines the channel.

7 EVALUATION

We evaluate five questions. **RQ1**: Do undefended workflows leak secrets through content-free usage records? **RQ2**: Does shaping remove the leak, and do weaker or deployed defenses? **RQ3**: What does shaping cost? **RQ4**: Does it preserve the workflow's function? **RQ5**: How strong is the evidence that the implementation meets the formal model, and does the validator catch compiler bugs?

7.1 Setup

Workflows and data. Table 2 lists the nine ports. Inputs come from public datasets at pinned revisions, sampled with a fixed seed and balanced over a secret attribute: DBpedia topics [61] for the joke and report workflows (entity category; person classes excluded because the models refuse to joke about real people) and for routing requests (requested form); a retail-intent corpus for the Prompt Flow switch (intent); synthetic English support tickets for the email agent (queue family); SQuAD [46] and SQuAD 2.0 [45] for the retrieval workflows (whether the question's article is in the indexed corpus, and whether it is answerable); and GSM8K [14] for the calculator agent (number of reference computation steps).

Models and execution. All calls use Llama 3.2 Instruct [36] in 4-bit GGUF quantization on four CPU cores with greedy decoding: the 1B model everywhere, and the 3B model for generation and query rewriting in adaptive RAG, mirroring the original's small-grader/large-generator split. A pilot with Qwen 2.5 1.5B was abandoned because it refused 14 of 20 joke requests about obscure entities, while Llama 3.2 1B refused none. Each input is run twice, undefended (*orig*) and shaped (*ours*), with the same run seed. In total the evaluation comprises 420 workflow executions over 210 inputs.

Attacker. The attacker sees exactly the usage records of section 2.1: per call the model, input and output token counts (and, in a separate experiment, latency). Features are the number of calls, total input and output tokens, and per-position model, input and output tokens. A class-balanced random forest (200 trees) is evaluated with 5-fold cross-validation, stratified when every class has five members. We report balanced accuracy, whose chance level is $1/\#\text{classes}$ regardless of class imbalance, and $I(Y;\hat{Y})$, the mutual information between the secret and the cross-validated prediction, which lower-bounds the information in the trace. Two secrets are attacked: the workflow's *internal decision*—the sequence of executed (non-dummy) call sites together with the route or verdict variable where the workflow has one—and the dataset's *secret attribute*.

Baselines. Besides the undefended workflow we evaluate three weaker defenses, derived exactly from the recorded runs: *pad-only* pads every call to its static (T, N) but neither aligns branches nor pads loops, generalizing the single-call output padding of Zhang et al. [60]; *random padding* adds uniformly random input and output tokens per call with the same mean total as shaping, in the style of deployed packet obfuscation [33, 41]; and *no-force* is the shaped program on an API that cannot force output length, so outputs keep their natural length. For cost we also compile two strategies that are secure by the same theorem: *naive* (every secret arm also executes the other

Table 2. Ported workflows. Sites: static model-call sites; lines: OrchLang source lines; findings: constructs reported by the detector; calls: model calls per undefended run (mean, range) and per shaped run (constant).

Workflow	Sites	Lines	Findings	Inputs	Calls (orig)	Calls (ours)
PF switch	1	18	1	210	1.0 (1–1)	1

Table 3. Balanced accuracy (%) of the usage-record attacker. Internal decision: path of executed call sites and route variable; attribute: the dataset's secret label. $I(Y;\hat{Y})$ in bits. Pad-only, random padding and no-force are derived from the recorded runs (section 7.1); *ours* emits a single trace in every workflow, so its exact mutual information is zero.

			Undefended		Weaker defenses			Shaped		Attribute	
Workflow	Cls.	Chance	Acc.	$I(Y;\hat{Y})$	Pad-only	Random	No-force	Acc.	I	Orig	Ours
PF switch	4	25	25	0.01	21	25	26	25	0.00	41	33

arms' calls as dummies, i.e., secure multi-execution [19]) and *uniform* (every call to a model is padded to one global cell size, as in BuFLO [21]).

7.2 RQ1: Undefended workflows leak

Table 3 reports the attacks. On undefended workflows the attacker recovers the internal decision with 25% median balanced accuracy, against a median chance level of 25%. The leak is strongest where branches differ in the number of calls (email agent, prompt chaining, evaluator–optimizer, adaptive RAG, the calculator agent's turn count): there the trace length alone identifies the path. It is weaker but present where only token counts differ (the Prompt Flow switch, corrective RAG), matching the census observation that constant call counts do not prevent leakage. The dataset attribute, which the workflow sees only through the model's behavior, is recovered with 41% median balanced accuracy.

7.3 RQ2: Shaping removes the leak; weaker defenses do not

Every shaped run of every workflow emitted the same usage trace as every other run of that workflow: across 210 shaped runs there is exactly one distinct trace per workflow, and each trace equals the static shape computed by the Lean validator (210 of 210 runs; the remainder, if any, is listed in the artifact). The attacker's balanced accuracy is therefore the chance level and the mutual information between the trace and any secret is exactly zero, as theorem 5.1 predicts.

The weaker defenses leave substantial leakage. Pad-only, which hides each call's lengths but not which calls happen, still leaks in 0 of 1 workflows (balanced accuracy more than five points above chance), namely all workflows whose paths differ in call count or model. Random padding at the same mean token budget leaves the call structure intact and only blurs counts; it leaks in 0 workflows. No-force, the strongest option on APIs that cannot force output length, aligns the call structure and pads inputs exactly but leaks output lengths, in 0 workflows. Exact output forcing is thus necessary, not merely convenient.

7.4 RQ3: Cost

Table 4 reports cost. Because a shaped workflow always runs its worst-case shape with every call at its bound, it costs 1.6× the undefended workflow at the median, in price units with output tokens at four times input tokens (range 1.6–1.6×). The overhead is dominated by forced outputs: output

Table 4. Cost per run. Undefended: mean calls, input and output tokens. Shaped: constant. Price ratio: shaped (or a static strategy) over undefended, with output tokens priced at 4× input. Latency: mean seconds per run on four CPU cores.

	Undefended			Shaped			Price ratio			Latency (s)	
Workflow	Calls	In	Out	Calls	In	Out	Ours	Naive	Uniform	Orig	Ours
PF switch	1.0	139	78	1	198	128	1.6	1.6	1.6	4.3	6.3

Table 5. Function. Accuracy (%) where ground truth exists, with exact McNemar p; agreement of the internal decision and of all stored outputs with the undefended run (%); requests truncated to their bound.

Workflow	Orig acc.	Ours acc.	p	Decision agr.	Output equal	Trunc.
PF switch	35	32	0.31	86	1	0

caps that the ports inherit or set generously become the actual output length of every call. It is largest for loops whose bound is rarely reached and for workflows whose undefended paths are usually short.

Alignment matters where secret arms differ. Compared with executing every arm's calls (naive), the minimum-cost alignment saves up to 0% (median 0%; in workflows where one arm's calls are a subsequence of another's, both strategies coincide). Compared with a single global cell size per model (uniform), it saves up to 0% (median 0%). Taint guidance pads every call in these nine ports, because every call depends on the user's secret message; on the 185 randomly generated programs of section 7.7 that mix public and secret calls, it lowers execution cost by 8% on average and by up to 91% relative to padding every call, and by nothing in programs where every expensive call is secret.

7.5 RQ4: Function

Forced decoding is transparent. For 60 greedy and 60 sampled ($T = 0.8$) generations from padded requests drawn from five workflows, with caps of 16–96 tokens and forced lengths 1–96 tokens beyond the cap, the visible output of forced decoding equalled natural decoding in 60 and 60 cases respectively, and every padded request and forced output had exactly the target token count (120 and 120 of 120). The output half of the Transparency hypothesis therefore holds exactly for this serving stack.

Input padding perturbs, but does not degrade, the task. Adding the padding message changes the prompt, so shaped workflows do not always reproduce the undefended outputs byte for byte (table 5). We measure the effect where ground truth exists. Task accuracy of shaped and undefended workflows differs by at most 2.9 points, and no difference is significant under an exact McNemar test at the 5% level. Internal decisions agree with the undefended run in most inputs; disagreements are perturbations in both directions rather than systematic degradation. A pilot on 80 news-topic classifications with Qwen 2.5 1.5B and 150 padding tokens found that the filler " ." changed 6.25% of decisions without reducing accuracy (81.25% vs. 80%), while " pad" changed 15% and " _" changed 17.5% and cost 12.5 accuracy points, so the filler is a tunable part of the design. Truncation of over-long requests occurred 0 times in all shaped runs.

7.6 Latency

Latency is outside the theorem but is visible to telemetry. Because a shaped workflow performs the same prefill and decoding work on every run, its latency becomes nearly constant: the coefficient of variation of per-run latency drops from 0.43 to 0.02 at the median. An attacker who also sees per-call latency recovers the internal decision with 27% median balanced accuracy on undefended workflows and 23% on shaped ones (maximum 23%). Residual latency variation stems from content-dependent compute (attention over different tokens, CPU scheduling) and is small but not zero; deployments that must also hide timing can add predictive timing mitigation [3, 59] on top of shaping, which is easier precisely because the work per call is fixed.

7.7 RQ5: Validation and fault injection

Differential testing. We generated 1000 random core programs with nested branches and loops, two to five variables, and random secret labellings, shaped each with all four strategies, and validated all 5000 original and shaped programs in Lean (6.8 seconds for the batch). Every shaped program was accepted; Python's mirror of the discipline and the leak detector agreed with Lean on every original program (649 originals were already secure). Executing every shaped program four times under pairs of different secret inputs with the mock backend produced identical traces in 16000 of 16000 pairs, traces equal to Lean's static shape whenever one was defined, and final stores equal to the original program's in 16000 of 16000 runs. This testing found two bugs during development, both now fixed and covered by the theorems: the compiler raised a merged call's forced length without capping its visible output (motivating the explicit cap k and `capsOK`), and the validator rejected correct programs whose sequences the compiler had re-associated (motivating `clean`).

Fault injection. To test whether the validator would catch realistic compiler bugs, we applied 11 mutation operators to the shaped versions of the nine workflows at up to six positions each: dropping or shrinking a call's padding, undercutting a cap, dropping a dummy, changing a dummy's model or size, unpadding a loop, swapping sequenced statements, changing a request, and reusing a call site. Each of the 201 mutants was validated in Lean and executed by an independent oracle—the runtime with a mock model whose responses drive the workflows' routing tests—under 41 secret inputs, recording whether traces differed. The validator rejected all 89 mutants that the oracle showed to leak. It accepted 18 mutants, all of which shrink the input padding of a call outside any secret branch; such a call still has a constant footprint, the oracle observed no leak, and accepting them is correct. The remaining rejected mutants changed the program's function (detected by erasure) or violated the cap or randomness-site checks without producing an observable leak in the oracle's finite exploration.

8 DISCUSSION AND LIMITATIONS

Cost and how to reduce it. Shaping executes every workflow at its worst-case shape, and the worst case is defined by the bounds the developer writes. In our ports most of the overhead is forced output: a call whose natural answer is three tokens but whose cap is 128 now always generates 128 tokens. The bounds are therefore the main knob. Caps can be tightened from profiling data, trading the probability of truncating a long answer against cost; loop bounds can be set to the iteration counts that matter in practice rather than generous defaults; and arms with very different costs can be restructured so that the expensive work is shared. The guarantee holds for any bounds the developer chooses, so tuning them never weakens security. Where an application can tolerate a quantified leak, a natural extension is to align arms only within groups, which reveals at most $\log_2$ of the number of groups [49] at a lower cost; we leave this to future work.

Hosted APIs. Exact forcing of output length requires serving-system support (`ignore_eos`, `min_tokens`), which open serving stacks provide but which we did not find documented for hosted commercial APIs. For hosted models OrchLang can still align call structure and pad inputs exactly; our no-force condition shows that this removes the call-count and model channels but leaves the output-length channel, so it should be described as a partial defense. Hidden reasoning tokens, which some hosted models bill without returning, are an additional unpaddable usage field.

Assumptions behind the theorem. Conformance—exact input padding and exact forced lengths—is checked by the runtime on every call and held in every run of the evaluation. It depends on knowing the provider's tokenizer and chat template exactly; for open models both ship with the weights, and our renderers reproduce the templates byte for byte. Prompt caching changes latency and, at some providers, adds cached-token counts to usage records [23]; a shaped deployment should disable cross-request caching or include cached counts in the observable trace and pad the shared prefix accordingly. The theorem treats tools as observable only by name; a tool with its own observable side effects (a network request whose size depends on secrets) needs its own padding. Finally, the guarantee concerns the trace of one execution; an observer who links many executions of the same user still learns nothing from shaped traces, because each is constant.

Latency. Shaping makes the work per call constant, which removes the coarse timing differences that dominate undefended workflows, but wall-clock time still varies with content-dependent computation and system noise. Timing-sensitive deployments can add predictive mitigation, which pads each call's completion to a schedule [3]; shaping makes such schedules tight because the work per call is known statically.

Language restrictions. OrchLang requires bounded loops and fan-out and expresses glue logic as declared pure tools. Twenty-two of the twenty-five looping workflows in the census have no bound in code, so porting them requires choosing one; we view this as clarifying rather than restricting, since an unbounded loop has an unbounded worst-case cost and an unbounded-capacity channel. Fully dynamic agents that decide at run time which of many tools to call can be expressed as a bounded loop over a fixed tool menu, but at a cost proportional to the menu.

Threats to validity. The ports deviate from their originals in documented ways (structured outputs become text, retrieval is local, loops are bounded), which changes absolute token counts. The models are small and quantized; larger models would change the routing decisions and their accuracy, but not the existence of the channel or the effect of shaping, which is determined by control structure and is exact. Inputs are drawn from public datasets that stand in for private data. The attacker is a single, reasonably strong learner; a stronger attacker could only increase the measured leakage of undefended and weakly defended workflows, and cannot gain anything from shaped traces, which are constant. Each input is run once per condition with greedy decoding; under sampling, undefended traces would vary more, while shaped traces are constant under any decoding.

9 RELATED WORK

Side channels of LLM services. Encrypted streaming responses leak per-token lengths [54] and packet-size and timing sequences that reveal conversation topics [34]; deployed mitigations add random padding to streamed messages [33, 41], and later work recovers content despite them [32, 58]. Output token count, observed through timing, reveals an LLM classifier's class and a translator's target language; Zhang et al. [60] propose, among other defenses, padding a call's output to the longest expected length. Efficiency mechanisms add further timing channels: speculative decoding [9, 53] and shared prompt or key–value caches [23, 55]. These works concern a single model call and a network or timing observer. We study workflows, where the leak is in which calls

happen, and the usage-record observer, which sees exact counts even when transport is protected; our pad-only and random-padding baselines generalize the single-call defenses above and are shown to be insufficient for workflows.

Metadata leakage in agents and routers. Traffic analysis fingerprints agents and their tool-use workflows [57] and multi-agent tasks on cloud LLM services [44], whose authors conclude that effective defenses must shape the whole traffic profile. Dangol [16] show that communication-graph metadata of agent protocols leaks task class and that message counts and sequence shape remain a residual channel under padding, closed by constant-rate cover traffic at the transport layer. Most closely, Chen [12] measures, on 1.7 million real requests, how logged LLM router decisions leak sensitive topics even with content logging disabled, surveys the logging defaults of eleven gateways and platforms, and evaluates parity post-processing of the router's decisions. Our work is complementary in three ways: it addresses whole multi-call workflows rather than a single routing decision, it preserves the workflow's decisions rather than changing them, and it provides a compile-time transformation with a machine-checked guarantee. SICC [56] compiles multi-agent workflows so that the *form* of messages exchanged between agents is independent of private state and proves a noninterference theorem for the message transcript; it does not address the number, model or token counts of the underlying LLM calls, which is our channel. The two compose.

Information-flow control for agents. CaMeL [18] and FIDES [15] use capability and taint tracking to stop prompt injection and explicit exfiltration; both explicitly leave control-flow side channels open, which is the gap we address for the usage observer. Language-based agent control [63] lists cost and resource types as a possible policy kind, and Agenra [48] gives a Lean-verified core for agent authorization; our validator follows the same methodology of a verified core with explicit assumptions, for a confidentiality hyperproperty. Static cost analysis of agent workflows [20, 25, 26] bounds or catalogs spending, which is a unary property; we need relational bounds whose equality across secret branches the shaping enforces. LLM programming systems such as LMQL [7], DSPy [27] and SGLang [62] structure prompts and decoding but make no confidentiality claims.

Constant-time and constant-resource programming. The idea of transforming secret branches so that both perform the same observable operations goes back to cross-copying [1] and the program-counter security model [38]; FaCT [10] is a DSL whose compiler rewrites secret control flow into constant-time code, and Ngo et al. [40] combine information flow with amortized resource analysis to verify and synthesize constant-resource programs. Raccoon [47] executes decoy paths, and secure multi-execution [19] runs a program once per security level. Relational cost analysis [13] and relational verification of side channels [2, 11] bound cost differences between runs, and verified compilers preserve constant-time and constant-resource properties [4, 5]. OrchLang carries this line of work to a new resource model—LLM calls whose cost is set by a tokenizer and stochastic generation, observed per call by telemetry—with padding mechanisms specific to it (tokenizer-exact filler, forced decoding, model-matched dummy calls) and with translation validation [39, 43] rather than a verified compiler.

Traffic-analysis defenses. Website-fingerprinting defenses pad traffic to a fixed rate and size (BuFLO [21], Tamaraw [8]) or, more cheaply, to supersequences over anonymity sets [52]; Pacer [35] shapes cloud tenants' traffic so its shape is secret-independent by construction, and Shrinkwrap [6] pads intermediate result sizes of query plans with differential privacy. Our uniform strategy corresponds to fixed-size cells and our alignment to anonymity-set supersequences, applied to the structure of a workflow rather than to packets.

10 CONCLUSION

Content redaction does not make LLM telemetry safe to share. When a workflow's control flow depends on private content—as it does in most of the example workflows its frameworks teach—the number of calls, their models and their token counts encode that content, and a simple classifier reads it back from usage records alone. OrchLang removes this channel by construction: it pads, forces and aligns exactly the calls whose usage could depend on secrets, and a Lean-verified validator guarantees that every accepted program emits the same usage trace for all secrets. The price is the cost of always running the worst case, which the developer controls through explicit bounds. We hope the census, the ports and the attack serve as a benchmark for future defenses, and that the observability ecosystem, which already distinguishes content from metadata, will start treating usage metadata as sensitive too.

ARTIFACT AVAILABILITY

The repository contains the OrchLang compiler and runtime, the Lean development and executable validator, the nine workflow ports with their specifications, the census data and scripts, dataset builders pinned to dataset revisions, all raw run records, and the scripts that generate every number and table in this paper from those records. Models are public Llama 3.2 Instruct GGUF files; no API key or GPU is required.

DECLARATIONS

Use of AI assistants. The implementation, proofs, experiment scripts and an initial draft of this paper were produced with substantial assistance from AI coding assistants (Anthropic Claude through Claude Code, and earlier OpenAI Codex), directed and reviewed by the authors. The authors take full responsibility for the content, have checked the proofs mechanically with Lean, and have verified the reported results against the released raw records.

Funding and competing interests. No external funding was received. The authors declare no competing interests.

REFERENCES

[1] Johan Agat. 2000. Transforming out Timing Leaks. In *Proceedings of the 27th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 40–53. https://doi.org/10.1145/325694.325702

[2] Timos Antonopoulos, Paul Gazzillo, Michael Hicks, Eric Koskinen, Tachio Terauchi, and Shiyi Wei. 2017. Decomposition Instead of Self-Composition for Proving the Absence of Timing Channels. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 362–375. https://doi.org/10.1145/3062341.3062378

[3] Aslan Askarov, Danfeng Zhang, and Andrew C. Myers. 2010. Predictive Black-Box Mitigation of Timing Channels. In *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS)*. 297–307. https://doi.org/10.1145/1866307.1866341

[4] Gilles Barthe, Sandrine Blazy, Benjamin Grégoire, Rémi Hutin, Vincent Laporte, David Pichardie, and Alix Trieu. 2020. Formal Verification of a Constant-Time Preserving C Compiler. *Proceedings of the ACM on Programming Languages* 4, POPL (2020). https://doi.org/10.1145/3371075

[5] Gilles Barthe, Sandrine Blazy, Rémi Hutin, and David Pichardie. 2021. Secure Compilation of Constant-Resource Programs. In *IEEE 34th Computer Security Foundations Symposium (CSF)*. https://doi.org/10.1109/CSF51468.2021.00020

[6] Johes Bater, Xi He, William Ehrich, Ashwin Machanavajjhala, and Jennie Rogers. 2018. Shrinkwrap: Efficient SQL Query Processing in Differentially Private Data Federations. *Proceedings of the VLDB Endowment* 12, 3 (2018), 307–320. https://doi.org/10.14778/3291264.3291274

[7] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2023. Prompting Is Programming: A Query Language for Large Language Models. In *Proceedings of the 44th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. https://doi.org/10.1145/3591300

[8] Xiang Cai, Rishab Nithyanand, Tao Wang, Rob Johnson, and Ian Goldberg. 2014. A Systematic Approach to Developing and Evaluating Website Fingerprinting Defenses. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 227–238. https://doi.org/10.1145/2660267.2660362

[9] Nicholas Carlini and Milad Nasr. 2024. Remote Timing Attacks on Efficient Language Model Inference. arXiv:2410.17175.

[10] Sunjay Cauligi, Gary Soeller, Brian Johannesmeyer, Fraser Brown, Riad S. Wahby, John Renner, Benjamin Grégoire, Gilles Barthe, Ranjit Jhala, and Deian Stefan. 2019. FaCT: A DSL for Timing-Sensitive Computation. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 174–189. https://doi.org/10.1145/3314221.3314605

[11] Jia Chen, Yu Feng, and Isil Dillig. 2017. Precise Detection of Side-Channel Vulnerabilities using Quantitative Cartesian Hoare Logic. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 875–890. https://doi.org/10.1145/3133956.3134058

[12] Teng-Ruei Chen. 2026. Sensitive-Topic Leakage Through LLM Routing Metadata: Measurement and Mitigation. arXiv:2610.09981.

[13] Ezgi Çiçek, Gilles Barthe, Marco Gaboardi, Deepak Garg, and Jan Hoffmann. 2017. Relational Cost Analysis. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*. 316–329. https://doi.org/10.1145/3009837.3009858

[14] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168.

[15] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2025. Securing AI Agents with Information-Flow Control. arXiv:2505.23643.

[16] Bijaya Dangol. 2026. From Privacy to Workflow Integrity: Communication-Graph Metadata in Autonomous Agent Interoperability. arXiv:2606.07150.

[17] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction – CADE 28 (LNCS, Vol. 12699)*. 625–635. https://doi.org/10.1007/978-3-030-79876-5_37

[18] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813.

[19] Dominique Devriese and Frank Piessens. 2010. Noninterference through Secure Multi-Execution. In *IEEE Symposium on Security and Privacy (S&P)*. 109–124. https://doi.org/10.1109/SP.2010.15

[20] Kevin Dickerson. 2026. Statically Bounding Resource Accumulation in LLM Agent Workflows: The Cost Case. Author-hosted report, https://loom.technology/research/bounded-llm-cost/. 18 May 2026; not peer reviewed.

[21] Kevin P. Dyer, Scott E. Coull, Thomas Ristenpart, and Thomas Shrimpton. 2012. Peek-a-Boo, I Still See You: Why Efficient Traffic Analysis Countermeasures Fail. In *IEEE Symposium on Security and Privacy (S&P)*. 332–346. https://doi.org/10.1109/SP.2012.28

[22] ggml-org. 2026. llama.cpp HTTP server: `n_predict` and `ignore_eos`. https://github.com/ggml-org/llama.cpp. Accessed 23 October 2026.

[23] Chenchen Gu, Xiang Lisa Li, Rohith Kuditipudi, Percy Liang, and Tatsunori Hashimoto. 2025. Auditing Prompt Caching in Language Model APIs. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (PMLR, Vol. 267)*. 20477–20496.

[24] Helicone. 2026. Omit Logs (Helicone documentation). https://docs.helicone.ai/features/advanced-usage/omit-logs. Accessed 23 October 2026.

[25] Hou, Wang, Zhao, and Wang. 2026. When Agents Do Not Stop: Uncovering Infinite Agentic Loops in LLM Agents. arXiv:2607.01641.

[26] Sajjad Khan. 2026. Token Budgets: An Empirical Catalog of 63 LLM-Agent Budget-Overrun Incidents, with an Affine-Typed Rust Mitigation as a Case Study. arXiv:2606.04056.

[27] Omar Khattab et al. 2024. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *International Conference on Learning Representations (ICLR)*.

[28] LangChain. 2026. LangChain and LangGraph documentation, revision `5d83d5e690af84ed654a8d4320bf9969af28e39f`. https://github.com/langchain-ai/docs.

[29] LangChain. 2026. LangGraph repository, examples, revision `eb066bd31a2bf01fc0cd58256df4800a3695c233`. https://github.com/langchain-ai/langgraph.

[30] LangChain. 2026. Prevent logging of sensitive data in traces (LangSmith documentation). https://docs.langchain.com/langsmith/mask-inputs-outputs. Accessed 23 October 2026.

[31] Langfuse. 2026. Masking (Langfuse documentation). https://langfuse.com/docs/observability/features/masking. Accessed 23 October 2026.

[32] Sijia Li, Cui, Chen, Lin, Gu, Deng, Xu, and Li. 2026. From Length to Content: Token-Length Side-Channel Attacks on LLM API Merged Outputs. In *35th USENIX Security Symposium*.

[33] Celso Martinho and Michelle Chen. 2024. Mitigating a token-length side-channel attack in our AI products. Cloudflare blog, https://blog.cloudflare.com/ai-side-channel-attack-mitigated.

[34] Geoff McDonald and Jonathan Bar Or. 2025. Whisper Leak: a side-channel attack on Large Language Models. arXiv:2511.03675.

[35] Aastha Mehta, Mohamed Alzayat, Roberta De Viti, Björn B. Brandenburg, Peter Druschel, and Deepak Garg. 2022. Pacer: Comprehensive Network Side-Channel Mitigation in the Cloud. In *31st USENIX Security Symposium*.

[36] Meta. 2024. Llama 3.2 1B and 3B Instruct model cards. https://huggingface.co/meta-llama.

[37] Microsoft. 2026. Prompt Flow repository, examples, revision `33a68c9d3cf349d7fdfad1f41969af27543c0406`. https://github.com/microsoft/promptflow.

[38] David Molnar, Matt Piotrowski, David Schultz, and David Wagner. 2006. The Program Counter Security Model: Automatic Detection and Removal of Control-Flow Side Channel Attacks. In *Information Security and Cryptology (ICISC 2005) (LNCS, Vol. 3935)*. https://doi.org/10.1007/11734727_14

[39] George C. Necula. 2000. Translation Validation for an Optimizing Compiler. In *Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation (PLDI)*. 83–94. https://doi.org/10.1145/349299.349314

[40] Van Chan Ngo, Mario Dehesa-Azuara, Matthew Fredrikson, and Jan Hoffmann. 2017. Verifying and Synthesizing Constant-Resource Implementations with Types. In *IEEE Symposium on Security and Privacy (S&P)*. 710–728. https://doi.org/10.1109/SP.2017.53

[41] OpenAI. 2026. API reference: `stream_options.include_obfuscation`. https://developers.openai.com/api/docs/api-reference/responses/get. Accessed 23 October 2026.

[42] OpenTelemetry. 2026. Semantic Conventions for Generative AI Spans (status: Development). https://github.com/open-telemetry/semantic-conventions-genai. Accessed 23 October 2026.

[43] Amir Pnueli, Michael Siegel, and Eli Singerman. 1998. Translation Validation. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (LNCS, Vol. 1384)*. 151–166. https://doi.org/10.1007/BFb0054170

[44] Pouryousef, Lopez, Nowmi, Kamol, Hossain, Tran, and Rahman. 2026. A Large-Scale Benchmark and Risk Assessment of Traffic Analysis Attacks on Cloud LLM Services. arXiv:2609.31877.

[45] Pranav Rajpurkar, Robin Jia, and Percy Liang. 2018. Know What You Don't Know: Unanswerable Questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*. 784–789.

[46] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ Questions for Machine Comprehension of Text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2383–2392.

[47] Ashay Rane, Calvin Lin, and Mohit Tiwari. 2015. Raccoon: Closing Digital Side-Channels through Obfuscated Execution. In *24th USENIX Security Symposium*. 431–446.

[48] Ammanaghatta Shivakumar, Priya, and Gao. 2026. Agenra: A Language and Verified Core for Stateful LLM Agent Authorization. arXiv:2608.22868.

[49] Geoffrey Smith. 2009. On the Foundations of Quantitative Information Flow. Foundations of Software Science and Computational Structures (FoSSaCS), LNCS 5504, pp. 288–302. https://doi.org/10.1007/978-3-642-00596-1_21

[50] vLLM project. 2026. `SamplingParams`: `min_tokens` and `ignore_eos` (`vllm/sampling_params.py`). https://github.com/vllm-project/vllm. Accessed 23 October 2026.

[51] Dennis Volpano, Cynthia Irvine, and Geoffrey Smith. 1996. A Sound Type System for Secure Flow Analysis. *Journal of Computer Security* 4, 2–3 (1996), 167–187. https://doi.org/10.3233/JCS-1996-42-304

[52] Tao Wang, Xiang Cai, Rishab Nithyanand, Rob Johnson, and Ian Goldberg. 2014. Effective Attacks and Provable Defenses for Website Fingerprinting. In *23rd USENIX Security Symposium*. 143–157.

[53] Jiankun Wei, Abdulrahman Abdulrazzag, Tianchen Zhang, Adel Muursepp, and Gururaj Saileshwar. 2024. When Speculation Spills Secrets: Side Channels via Speculative Decoding in LLMs. arXiv:2411.01076.

[54] Roy Weiss, Daniel Ayzenshteyn, Guy Amit, and Yisroel Mirsky. 2024. What Was Your Prompt? A Remote Keylogging Attack on AI Assistants. In *33rd USENIX Security Symposium*. 3367–3384.

[55] Guanlong Wu, Zheng Zhang, Yao Zhang, Weili Wang, Jianyu Niu, Ye Wu, and Yinqian Zhang. 2025. I Know What You Asked: Prompt Leakage via KV-Cache Sharing in Multi-Tenant LLM Serving. In *Network and Distributed System Security Symposium (NDSS)*.

[56] Jinghan Xu, Longze Fan, Zeyuan Wang, Xinjin Li, and Hankai Liu. 2026. Selection-Invariant Communication Compilers for Privacy-Aware Multi-Agent LLM Workflows. arXiv:2609.26076.

[57] Zhang, Deng, Gu, Chen, Xu, Li, and Wu. 2025. Exposing LLM User Privacy via Traffic Fingerprint Analysis: A Study of Privacy Risks in LLM Agent Interactions. arXiv:2510.07176.

[58] Zhang, Wu, Deng, Wang, and Zhang. 2025. NetEcho: From Real-World Streaming Side-Channels to Full LLM Conversation Recovery. arXiv:2510.25472.

[59] Danfeng Zhang, Aslan Askarov, and Andrew C. Myers. 2012. Language-Based Control and Mitigation of Timing Channels. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 99–110. https://doi.org/10.1145/2254064.2254078
[60] Tianchen Zhang, Gururaj Saileshwar, and David Lie. 2024. Time Will Tell: Timing Side Channels via Output Token Count in Large Language Models. arXiv:2412.15431.
[61] Xiang Zhang, Junbo Zhao, and Yann LeCun. 2015. Character-level Convolutional Networks for Text Classification. In *Advances in Neural Information Processing Systems (NeurIPS)*.
[62] Lianmin Zheng et al. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems (NeurIPS)*.
[63] Zhou, Loris D'Antoni, and Nadia Polikarpova. 2026. Language-Based Agent Control. arXiv:2605.12863.

A PORTED WORKFLOWS

Each port's header comment names its source file and line range at the pinned revision; the artifact's `experiments/census/ports/` directory contains a specification of each with the verbatim source excerpts, node graph, routing conditions and a list of deviations. Table 6 summarizes them.

Table 6. Sources, secret attributes and principal deviations of the ports. All ports add output caps and replace the original models with Llama 3.2.

Workflow	Source (pinned)	Secret attribute	Deviations
Routing	LangGraph docs, *workflows-agents*, routing	requested form (story/joke/poem)	structured `Route` output as one word; branch system prompts from the JavaScript variant
Prompt chaining	same, prompt chaining (Graph API gate)	DBpedia entity class	none beyond caps
Evaluator–optimizer	same, evaluator–optimizer	DBpedia entity class	loop bounded at 3 rounds; `Feedback` schema as two text lines
Orchestrator–worker	same, orchestrator–worker (`Send`)	DBpedia entity class	fan-out bounded at 4 and sequential; sections as text lines
PF switch	Prompt Flow *conditional-flow-for-switch*	retail intent	none beyond caps; Jinja roles as chat messages
Email agent	LangGraph docs, *thinking-in-langgraph*	ticket queue family	classification schema as four text lines; human review becomes a fixed reply
Corrective RAG	LangGraph *langgraph_crag_local.ipynb*	question in indexed corpus	BM25 over SQuAD paragraphs for vector store and web search; passages capped at 230 tokens
Calculator agent	LangGraph docs, agents (calculator tools)	GSM8K step count	native tool calling as a CALL/FINAL text protocol; history serialized into the user turn; 6 turns
Adaptive RAG	LangGraph *langgraph_adaptive_rag.ipynb*	in corpus / answerable	router prompt names the indexed titles; cycles bounded (2 retrieval, 2 generation rounds); "not useful" retries generation

B LEAN DEVELOPMENT

Table 7 maps the paper's definitions and theorems to the Lean development. The file checks with Lean 4.19.0 in about three seconds; `#print axioms` reports `propext` and `Quot.sound` for every main theorem, and `erase_correct` uses only `propext`.

Table 7. Correspondence between the paper and `proof/ShapeCore.lean`.

Paper	Lean
World, Conformance, Transparency	`World`, `World.Conforms`, `World.Transparent`
Execution, static shape	`exec`, `loopRun`, `shape`
Padding discipline, randomness sites, caps	`wt`, `sitesOK`, `capsOK`
Shape lemma, confinement	`shape_exec`, `high_store`, `high_count`
Theorem 5.1 and corollary	`noninterference`, `trace_noninterference`, `distribution_noninterference`
Theorem 5.3	`erase_correct`, `clean_data`
Validator and its soundness	`validateProgram`, `validateProgram_sound`
Executable checker	`checkJson`, `main`